

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: David Gregorio López Pita

PARALELO: "A"

SEMESTRE: 5to. Nivel

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)"

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

1. Análisis del problema

Descripción del caso de estudio

Un Data Center requiere mantener operativos sus servidores, equipos de red, sistemas de almacenamiento, aplicaciones y servicios digitales. Sin embargo, pueden presentarse incidentes técnicos que afecten el funcionamiento normal de la infraestructura, tales como fallos de conectividad, errores en servidores, problemas de hardware, caídas de servicios o fallos de software.

Actualmente, cuando estos incidentes no se gestionan de forma ordenada, pueden generarse problemas como pérdida de información, retraso en la atención, duplicidad de reportes, ausencia de responsables técnicos, falta de seguimiento y escasa trazabilidad de las soluciones aplicadas.

Por esta razón, se propone el diseño de un sistema Help Desk que permita registrar, clasificar, asignar, atender y cerrar incidentes técnicos de forma organizada. El sistema deberá contar con actores definidos, flujo de atención, control de estados, notificaciones automáticas y una arquitectura basada en patrones de diseño.

Actores principales del sistema

Actor	Descripción	Funciones principales
Usuario final	Persona que reporta un problema técnico relacionado con los servicios del Data Center.	Crear ticket, describir el problema, consultar el estado del incidente.
Técnico de soporte	Especialista encargado de revisar, atender y resolver incidentes técnicos.	Recibir tickets, diagnosticar fallos, actualizar estados y registrar soluciones.
Administrador de sistemas	Responsable de supervisar el funcionamiento general del Help Desk.	Asignar técnicos, gestionar usuarios, revisar reportes y controlar el flujo de atención.
Sistema Help Desk	Plataforma encargada de procesar, registrar y notificar los incidentes.	Crear tickets, clasificar incidentes, enviar notificaciones y mantener historial.

Requerimientos funcionales y no funcionales

Requerimientos funcionales:

Código	Requerimiento funcional
RF01	El sistema debe permitir que el usuario final registre un ticket de incidente.
RF02	El sistema debe permitir clasificar el incidente como hardware, red o software.
RF03	El sistema debe permitir asignar un técnico especializado al incidente.
RF04	El sistema debe permitir actualizar el estado del ticket.
RF05	El sistema debe enviar una notificación automática al técnico asignado.
RF06	El sistema debe permitir consultar el historial de incidentes.
RF07	El sistema debe permitir registrar la solución aplicada por el técnico.
RF08	El sistema debe permitir cerrar el ticket una vez resuelto el incidente.



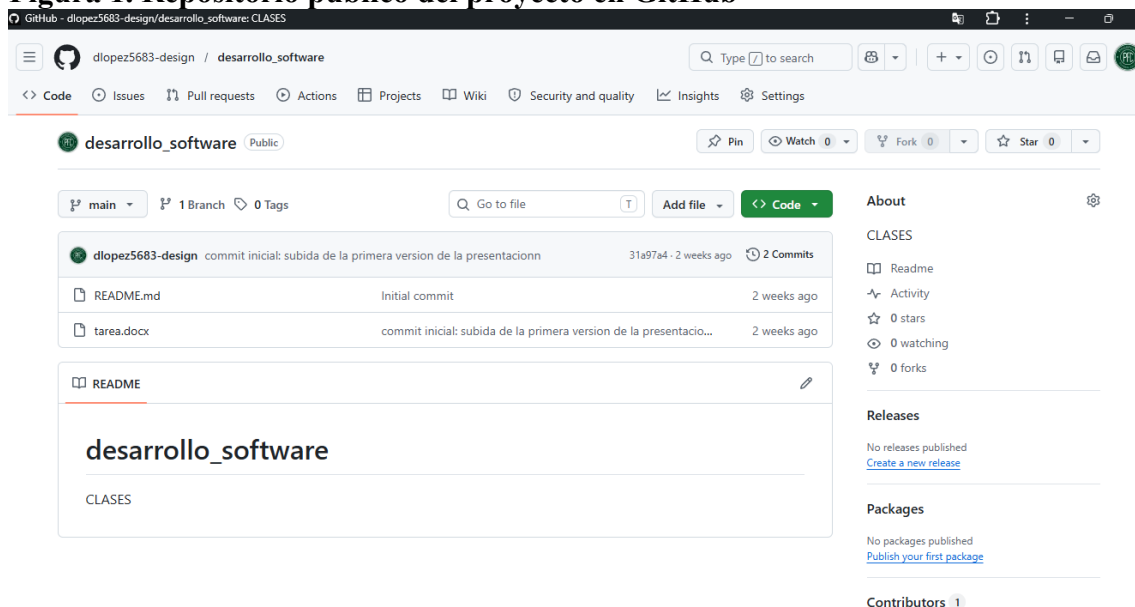
RF09	El sistema debe permitir generar reportes de incidentes atendidos, pendientes y cerrados.
RF10	El sistema debe permitir al administrador gestionar usuarios, técnicos y categorías de incidentes.

Requerimientos no funcionales

Código	Requerimiento no funcional
RNF01	El sistema debe ser escalable para permitir la incorporación de nuevos tipos de incidentes.
RNF02	El sistema debe ser mantenible mediante una arquitectura modular.
RNF03	El sistema debe garantizar trazabilidad del desarrollo mediante GitHub y Gitflow.
RNF04	El sistema debe separar la lógica de negocio, la interfaz y el acceso a datos.
RNF05	Los diagramas UML deben ser legibles, coherentes y relacionados con el problema planteado.
RNF06	El diseño debe aplicar principios SOLID para favorecer la reutilización y bajo acoplamiento.
RNF07	El sistema debe facilitar el seguimiento del ciclo de vida de cada ticket.
RNF08	El sistema debe permitir futuras integraciones con correo electrónico, bases de datos o aplicaciones móviles.

2. Diseño del sistema (Repositorio público de GitHub)

Figura 1. Repositorio público del proyecto en GitHub



En la figura se evidencia el repositorio público creado en GitHub para el desarrollo de la Actividad 1. El repositorio contiene los archivos iniciales del proyecto y permite mantener la trazabilidad del trabajo mediante control de versiones.



Figuras 2.1. Ramas creadas bajo el modelo Gitflow

```
MINGW64:/c:/Users/Aorus Gaming/Desktop/desarrollo_software
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (actividad-1)
$ ^[[200~git checkout main
git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1bash: $'\E[200~git': command not found
git pull origin main

git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1git pull origin main

git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (actividad-1)
$ git pull origin main
From https://github.com/dlopez5683-design/desarrollo_software
 * branch      main      -> FETCH_HEAD
Already up to date.

git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (actividad-1)
$
git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1git checkout -b develop
git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
```

Figuras 2.2. Ramas creadas bajo el modelo Gitflow

```
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (actividad-1)
$ git checkout -b develop
Switched to a new branch 'develop'
git push -u origin develop

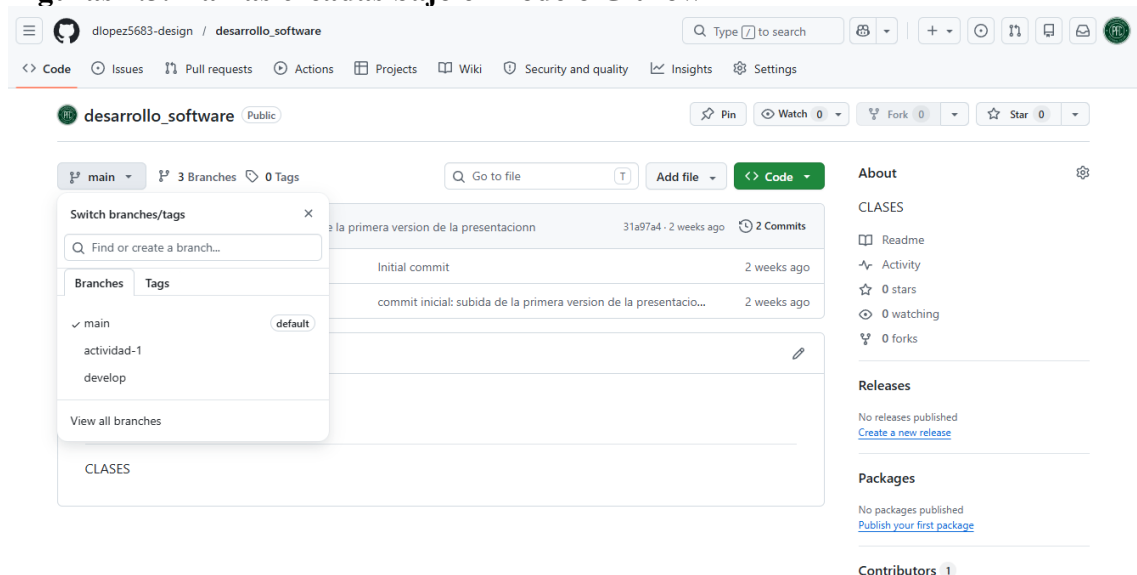
git checkout -b feature/actividad-1
git push -u origin feature/actividad-1git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1git push -u origin develop

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (develop)
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/dlopez5683-design/desarrollo_software/pull/new/develop
remote:
To https://github.com/dlopez5683-design/desarrollo_software.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.

git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (develop)
$
git checkout -b feature/actividad-1
git push -u origin feature/actividad-1git checkout -b feature/actividad-1
git push -u origin feature/actividad-1
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (develop)
$ git checkout -b feature/actividad-1
Switched to a new branch 'feature/actividad-1'
git push -u origin feature/actividad-1git push -u origin feature/actividad-1git push -u origin
feature/actividad-1
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$
```

Figuras 2.3. Ramas creadas bajo el modelo Gitflow



En la figura se observan las ramas del repositorio utilizadas para gestionar el desarrollo de la actividad: main, develop y ramas de trabajo para la implementación de avances, evidenciando la organización del proyecto mediante control de versiones.

Figura 3. Evidencia de ramas y commits del repositorio.

```
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git log --oneline --graph --decorate --all
* 31a97a4 (HEAD -> feature/actividad-1, origin/main, origin/develop, origin/actividad-1, origin/HEAD, main, develop, actividad-1) commit inicial: subida de la primera version de la presentacion
* 7ae39ec Initial commit

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$
```

La figura muestra el registro de commits asociados al desarrollo de la actividad, evidenciando el uso de control de versiones mediante Git y GitHub.

Figura 4.1 Archivos del repositorio en GitBash.

```
Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git status
On branch feature/actividad-1
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    diagrama UML.png
    diagrama de casos de usos.png
    diagrama de secuencia.png

nothing added to commit but untracked files present (use "git add" to track)

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git add .

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git status
On branch feature/actividad-1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   diagrama UML.png
    new file:   diagrama de casos de usos.png
    new file:   diagrama de secuencia.png

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git commit -m "subida de archivos de la actividad 1"
[feature/actividad-1 aaa8edd] subida de archivos de la actividad 1
3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 diagrama UML.png
 create mode 100644 diagrama de casos de usos.png
 create mode 100644 diagrama de secuencia.png

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git push
fatal: The current branch feature/actividad-1 has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin feature/actividad-1

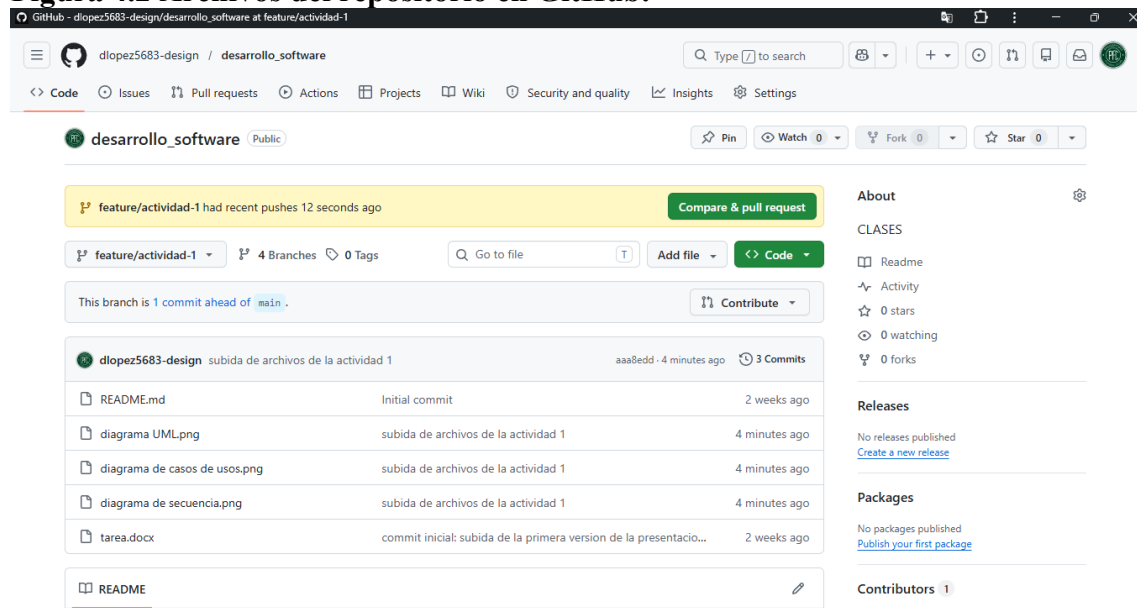
To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$ git push -u origin feature/actividad-1
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 703.63 KiB | 17.59 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/actividad-1' on GitHub by visiting:
remote:   https://github.com/dlopez5683-design/desarrollo_software/pull/new/feature/actividad-1
remote:
To https://github.com/dlopez5683-design/desarrollo_software.git
 * [new branch]      feature/actividad-1 -> feature/actividad-1
branch 'feature/actividad-1' set up to track 'origin/feature/actividad-1'.

Aorus Gaming@DESKTOP-V4MHS8V MINGW64 ~/Desktop/desarrollo_software (feature/actividad-1)
$
```



Figura 4.2 Archivos del repositorio en GitHub.



En la figura se evidencia la estructura de archivos cargados en el repositorio público de GitHub. Se observan los diagramas UML correspondientes al sistema Help Desk, el archivo de documentación de la actividad y el archivo README. Además, la captura muestra que el trabajo se encuentra en la rama feature/actividad-1, lo que permite evidenciar el uso de control de versiones y organización mediante ramas de desarrollo.

Diagrama de casos de uso

Actor	Casos de uso
Usuario final	Crear ticket, consultar estado del ticket.
Técnico de soporte	Revisar ticket asignado, actualizar estado, registrar solución.
Administrador de sistemas	Asignar técnico, gestionar usuarios, generar reportes, cerrar ticket.
Sistema Help Desk	Clasificar incidente, notificar técnico, almacenar historial.

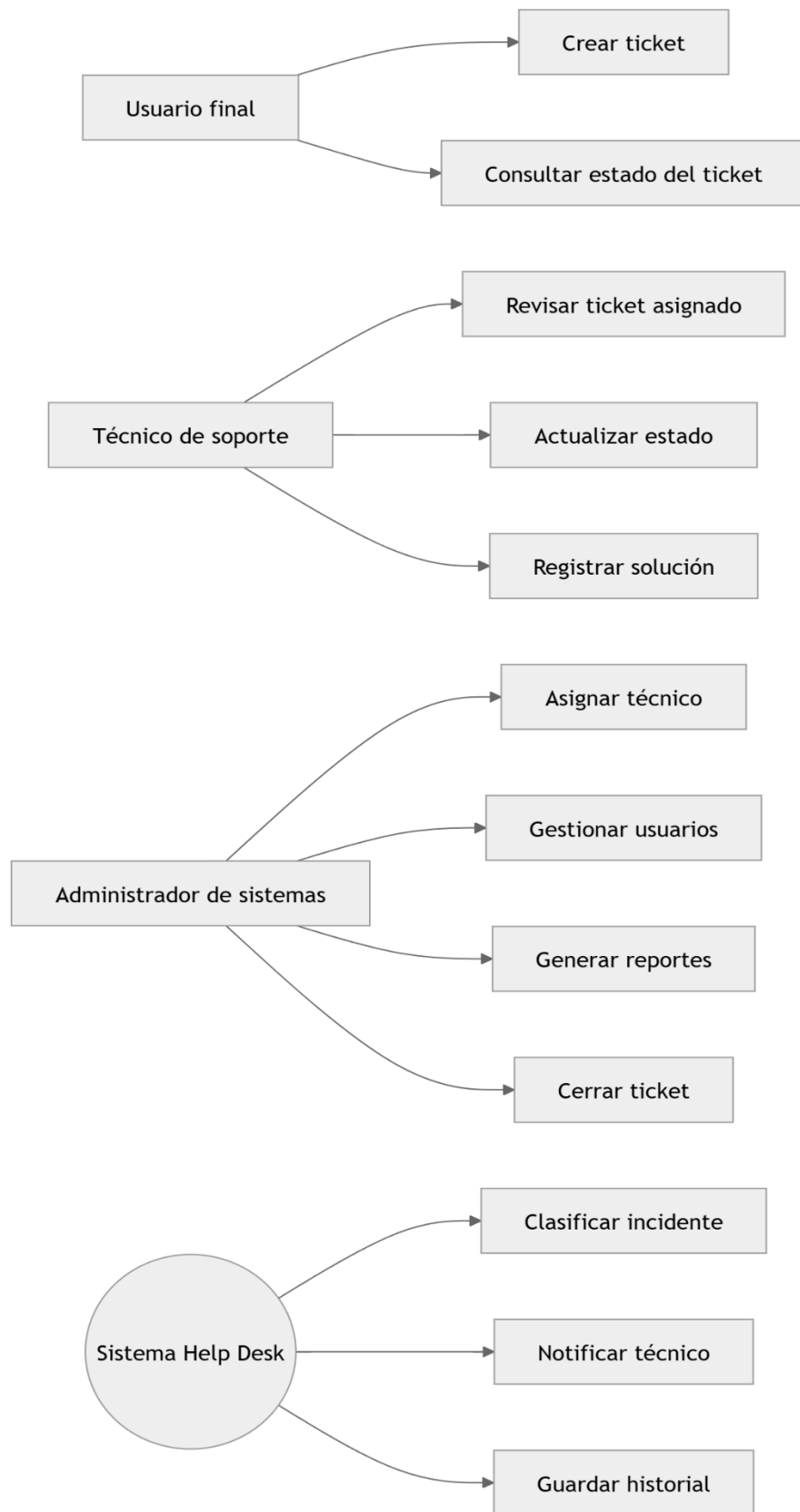




Diagrama de clases

El diagrama de clases representa la estructura lógica del sistema. Se proponen clases relacionadas con usuarios, tickets, incidentes, técnicos, administrador, gestor de tickets, fábrica de incidentes y sistema de notificaciones.

Clase	Descripción
Usuario	Representa a la persona que reporta un incidente.
Tecnico	Representa al responsable de atender el incidente.
Administrador	Gestiona usuarios, técnicos y asignaciones.
Ticket	Representa el reporte generado por un incidente.
Incidente	Clase base para los diferentes tipos de incidentes.
IncidenteHardware	Representa fallos físicos o de equipos.
IncidenteRed	Representa fallos de conectividad o infraestructura de red.
IncidenteSoftware	Representa fallos en aplicaciones, sistemas o servicios.
GestorTickets	Clase central encargada de registrar, asignar y controlar tickets.
IncidenteFactory	Crea objetos de incidentes según su tipo.
Notificador	Envía alertas a técnicos y usuarios.

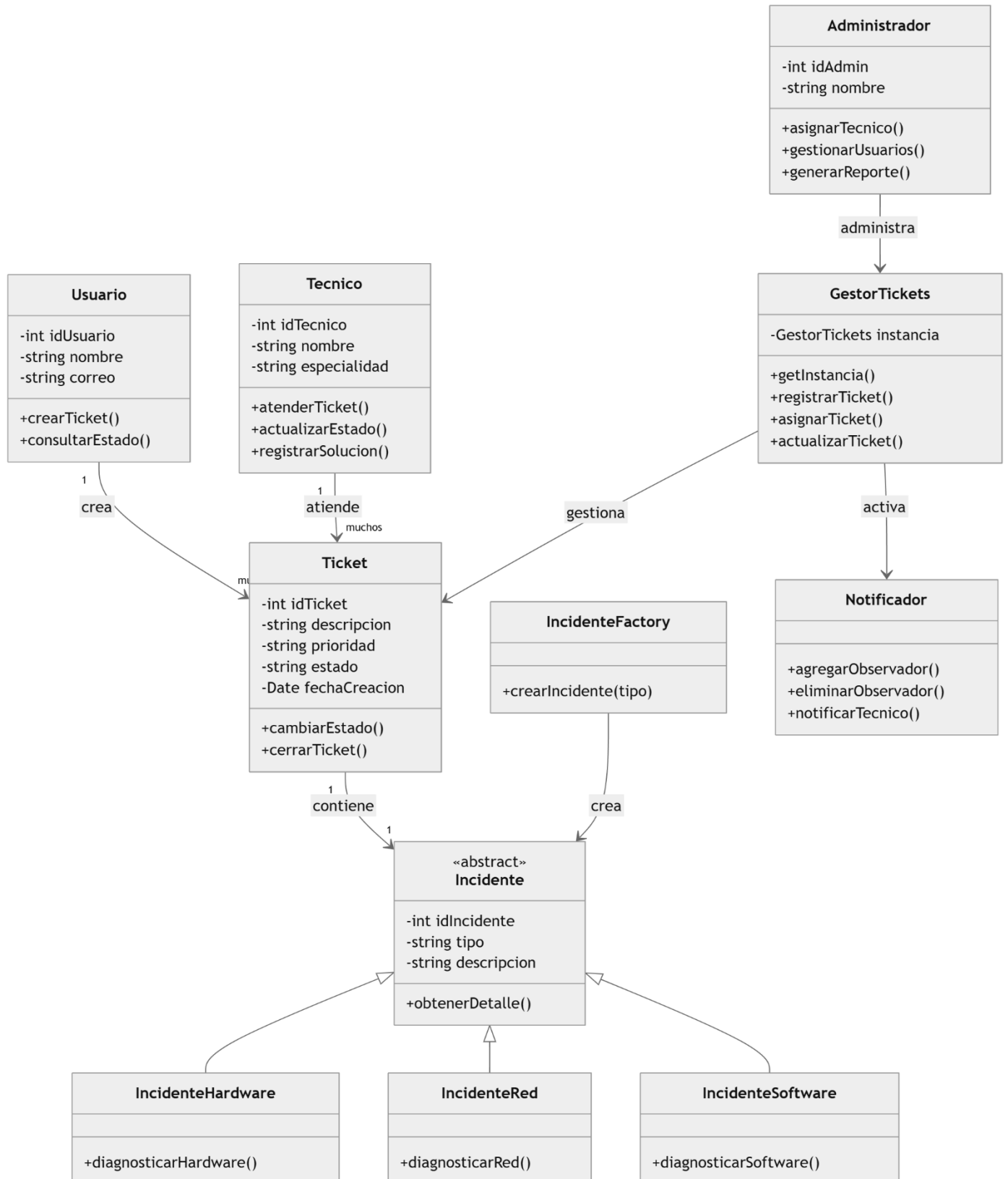


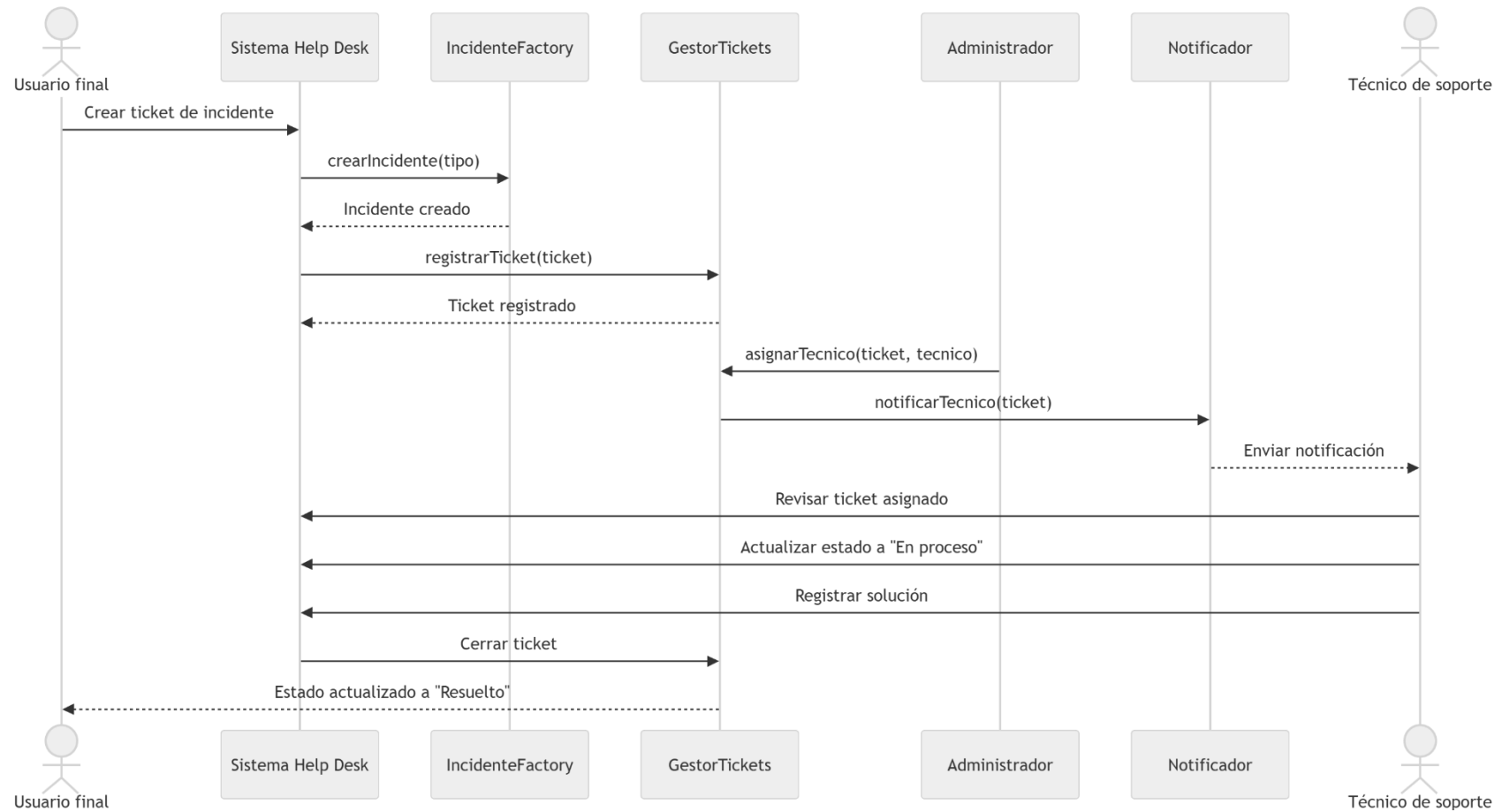


Diagrama de secuencia

El diagrama de secuencia muestra el flujo de interacción cuando un usuario reporta un incidente y el sistema lo asigna a un técnico.

Flujo del proceso

1. El usuario final ingresa al sistema.
2. El usuario crea un ticket.
3. El sistema solicita a la fábrica de incidentes crear el tipo correspondiente.
4. El gestor de tickets registra el ticket.
5. El administrador asigna un técnico.
6. El sistema notifica al técnico.
7. El técnico revisa el ticket.
8. El técnico actualiza el estado.
9. El técnico registra la solución.
10. El ticket se cierra.





Arquitectura general del sistema

El sistema Help Desk se propone bajo una arquitectura por capas, integrando el patrón MVC y componentes específicos para la gestión de tickets, creación de incidentes y notificaciones automáticas.

Capas del sistema

Capa	Componentes	Función
Capa de presentación	Vista de usuario, vista de técnico, vista de administrador	Permite la interacción entre los actores y el sistema.
Capa de controladores	TicketController, UsuarioController, NotificacionController	Coordina las acciones solicitadas por los usuarios.
Capa de modelo	Usuario, Técnico, Administrador, Ticket, Incidente	Representa las entidades principales del sistema.
Capa de patrones	GestorTickets, IncidenteFactory, Notificador	Implementa los patrones Singleton, Factory Method y Observer.
Capa de datos	Base de datos de usuarios, tickets e historial	Almacena la información del sistema.
Capa de versionamiento	GitHub, ramas master, develop y feature	Permite controlar los cambios del proyecto mediante Gitflow.

Aplicación de patrones de diseño

Patrón Singleton

Propósito: El patrón Singleton permite garantizar que una clase tenga una única instancia en todo el sistema. En este caso, se aplica en la clase GestorTickets, porque el sistema debe contar con un único componente central responsable de registrar, asignar, actualizar y cerrar tickets.

Aplicación en el sistema: La clase GestorTickets administra todos los tickets generados dentro del Help Desk. Al usar Singleton, se evita que existan múltiples gestores independientes que puedan generar duplicidad, inconsistencias o pérdida de trazabilidad. Problema que resuelve: Evita que varios objetos gestionen tickets de manera separada. Esto permite mantener un control centralizado del flujo de incidentes.

Justificación técnica: El patrón Singleton es adecuado porque el Help Desk necesita una instancia centralizada para controlar el ciclo de vida de los tickets. Esto mejora la coherencia del sistema y reduce riesgos de duplicidad en la gestión de incidentes.

Patrón Observer

Propósito: El patrón Observer permite que un objeto notifique automáticamente a otros objetos cuando ocurre un cambio importante. En este sistema se utiliza para notificar a los técnicos cuando se crea o asigna un nuevo ticket.

Aplicación en el sistema: Cuando el administrador asigna un ticket a un técnico, el sistema activa el componente Notificador, que envía una alerta al técnico correspondiente.



Problema que resuelve: Evita que el técnico tenga que revisar manualmente el sistema para saber si tiene nuevos tickets asignados. También permite automatizar el proceso de comunicación.

Justificación técnica: El patrón Observer es útil porque permite implementar un sistema reactivo de notificaciones. Cada vez que cambia el estado de un ticket o se asigna un responsable, los usuarios interesados pueden ser informados automáticamente.

Patrón Factory Method

Propósito: El patrón Factory Method permite crear objetos sin especificar directamente la clase concreta que se va a instanciar. En este caso, se utiliza para crear incidentes según su tipo: hardware, red o software.

Aplicación en el sistema: La clase IncidenteFactory recibe el tipo de incidente y crea el objeto correspondiente:

- ✓ Si tipo = Hardware → crear IncidenteHardware
- ✓ Si tipo = Red → crear IncidenteRed
- ✓ Si tipo = Software → crear IncidenteSoftware

Problema que resuelve: Evita que el sistema principal tenga que conocer los detalles de creación de cada tipo de incidente. Esto facilita agregar nuevos tipos de incidentes en el futuro.

Justificación técnica: Factory Method mejora la extensibilidad del sistema. Si en el futuro se desea agregar un nuevo tipo de incidente, por ejemplo, “Seguridad informática”, solo se crea una nueva clase sin modificar de forma significativa el resto del sistema.

Patrón MVC

Propósito: El patrón MVC, Modelo-Vista-Controlador, permite separar la aplicación en tres componentes: modelo, vista y controlador. Esta separación facilita el mantenimiento, la organización del código y la evolución del sistema.

Aplicación en el sistema:

Componente MVC	Aplicación en el Help Desk
Modelo	Clases Usuario, Técnico, Ticket, Incidente y Administrador.
Vista	Interfaces para usuario final, técnico y administrador.
Controlador	TicketController, UsuarioController y NotificacionController.

Problema que resuelve: Evita mezclar la interfaz del usuario con la lógica de negocio y los datos. Esto permite modificar una parte del sistema sin afectar directamente a las demás.

Justificación técnica: MVC es adecuado porque permite estructurar el Help Desk de forma clara y mantenible. La interfaz puede cambiar sin alterar los modelos, y la lógica de negocio puede ajustarse sin modificar directamente las vistas.

Principios SOLID aplicados

El diseño propuesto también se relaciona con principios SOLID, especialmente en la separación de responsabilidades y extensibilidad del sistema.



Principio	Aplicación en el sistema
S - Responsabilidad única	Cada clase tiene una función específica: Ticket gestiona datos del ticket, Notificador envía alertas, GestorTickets administra tickets.
O - Abierto/Cerrado	El sistema puede incorporar nuevos tipos de incidentes sin modificar toda la estructura.
L - Sustitución de Liskov	Las clases IncidenteHardware, IncidenteRed e IncidenteSoftware pueden utilizarse como variantes de la clase Incidente.
I - Segregación de interfaces	Cada actor accede únicamente a las funciones que necesita.
D - Inversión de dependencias	El sistema puede depender de abstracciones como Incidente y no directamente de clases concretas.

Bibliografía

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress.
- Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.